Introdução: Análise Estática Avançada (Engenharia Reversa)

Enquanto a análise básica foca em metadados, a Análise Estática Avançada mergulha na lógica interna do binário sem executá-lo. O objetivo é desconstruir o código de máquina para entender como o malware opera, contornando ofuscações e técnicas de evasão.

Nesta etapa, utilizamos desassembladores para traduzir bits em linguagem Assembly, permitindo mapear o comportamento real da ameaça.

Objetivos Principais:

- Mapear o Fluxo de Controle: Visualizar as tomadas de decisão do malware (Gráficos de Fluxo).
- Identificar Chamadas de Sistema (APIs): Rastrear como o código interage com o Windows.
- Vencer a Evasão: Localizar rotinas de Anti-VM e Anti-Debugging no código.
- Desofuscação: Revelar strings e IPs que permanecem cifrados em análises superficiais.

Ferramentas de Referência:

- IDA Pro / Ghidra: Desassemblagem e descompilação.
- Binary Ninja: Análise de fluxo de código.
- Cutter (Radare2): Engenharia reversa open-source.

Ghidra

O **Ghidra** é a ferramenta de código aberto da NSA que revolucionou a análise de malware. Sua maior força não é apenas mostrar o **Assembly** (linguagem de baixo nível), mas sim o seu **Descompilador**, que traduz instruções complexas de processador de volta para algo muito parecido com a linguagem **C**.

1. Fluxo de Análise no Ghidra

Para analisar qualquer amostra (como o `HelloWorld.exe` ou os `malbusters`), siga este padrão:

1. **New Project:** Crie um projeto "Non-Shared".
2. **Import:** Arraste o binário para dentro do Ghidra.
3. **Analyze:** Ao abrir o CodeBrowser, clique em **Yes** para analisar. Mantenha as opções padrão (elas identificam funções e strings automaticamente).
4. **The Wait:** Aguarde a barra de progresso no canto inferior direito chegar a 100%.

2. Dominando o Layout

Para um analista do SOC, estas três janelas são o seu "mapa do tesouro":

- **Symbol Tree (Árvore de Símbolos):** Vá em `Functions` e procure por `entry` ou `main`. É aqui que o código do autor começa de verdade.
- **Listing (Listagem):** Onde o código **Assembly** reside. Aqui você vê os *Opcodes* e as instruções (`PUSH`, `CALL`, `RET`).
- **Decompiler (Descompilador):** A janela mágica à direita. Ela transforma o Assembly difícil em código C legível. **Dica:** Se você clicar em uma linha no Assembly, o Ghidra destaca a linha correspondente no Descompilador.

3. Localizando Comportamentos (Exemplo: HelloWorld)

No caso do `HelloWorld.exe`, o objetivo é encontrar a função que exibe a mensagem:

1. **Busca de Strings:** Vá em `Search -> For Strings`. Procure por "Hello World".
2. **X-Ref (Referência Cruzada):** Ao clicar na string, você pode ver qual parte do código a utiliza.
3. **A Chamada:** Você verá o código preparando os argumentos e chamando a API `MessageBoxA`.

> **Conceito Chave:** No Assembly, antes de uma função ser chamada (CALL), o malware "empurra" (**PUSH**) os dados para a memória (pilha). Se você vê um `PUSH` de um IP ou URL seguido de um `CALL` para uma função de rede, você acabou de descobrir o destino do malware.

Guia de Tradução: C para Assembly

Para analisar os arquivos `if-else.exe` e `while-loop.exe`, foque nestes padrões de instruções:

1. O "Coração" das Condições (If-Else e Loops)

Toda decisão no Assembly geralmente depende de duas instruções trabalhando juntas:

- **`CMP` (Compare):** Compara dois valores (ex: o contador do loop vs. o limite).
- **`JMP` / `JZ` / `JNZ` / `JGE` (Jumps):** Saltos condicionais. Se a comparação for verdadeira, o programa "pula" para outra parte do código.

2. Estrutura de um Loop (For/While)

No Ghidra, você verá uma seta voltando para cima no gráfico de fluxo.

- **Inicialização:** Um registrador (como `EAX` ou `ECX`) recebe 0 ou 1.
- **Corpo do Loop:** As instruções que se repetem (como o `printf`).

- **Incremento:** Instrução `INC` ou `ADD` para aumentar o contador.
- **Verificação:** Um `CMP` seguido de um salto de volta ao início do loop.

3. Estrutura de If-Else

Diferente do loop, as setas no gráfico de fluxo do Ghidra não voltam; elas se dividem em dois caminhos e depois se reúnem.

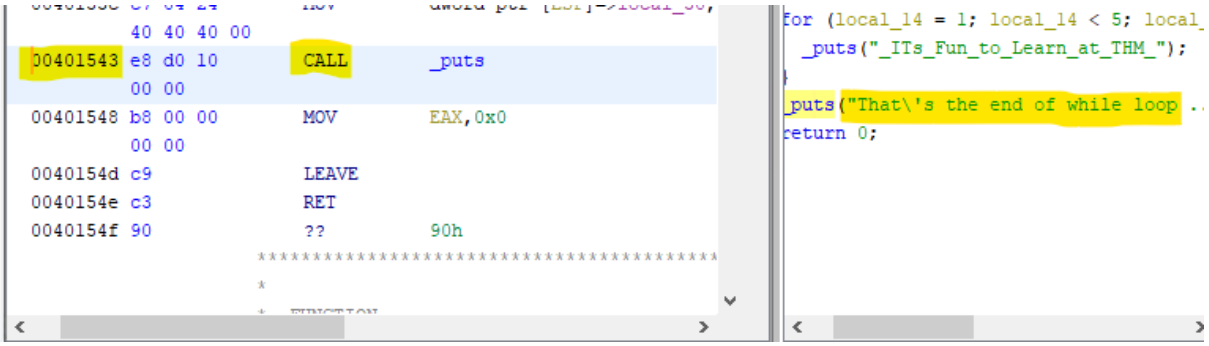
- Se a condição for falsa, o código pula o bloco "If" e vai direto para o "Else" ou para o fim da função.

Análise de Fluxo e Estruturas de Código (Ghidra)

1. No programa if-else.exe, examine as strings e complete a frase "This program demonstrates.....".

Resposta: 2. Examine o arquivo while-loop.exe no Ghidra. Qual é o endereço virtual da instrução que CALL a impressão da frase " That's the end of while loop .. "?

Resposta: 00401543



Introdução: Interação com o Sistema (Windows API Analysis)

Para realizar ações maliciosas, um binário deve interagir com o Sistema Operacional através de **APIs (Application Programming Interfaces)**. Na engenharia reversa, identificar quais APIs são chamadas e quais argumentos são passados é fundamental para mapear as capacidades do malware.

Um exemplo crítico é a função `CreateProcessA`. Enquanto um software legítimo a utiliza para abrir um editor de texto, um malware pode usá-la para realizar **Process Injection** ou executar um *payload* secundário de forma oculta.

O que observar no Assembly:

Ao analisar chamadas de API no Ghidra ou IDA, o foco deve estar na preparação da pilha (**Stack**). No padrão de chamada `stdcall` (comum no Windows x86), os argumentos são passados através da instrução `PUSH` em ordem inversa antes da instrução `CALL`.

APIs de Alto Interesse para o SOC:

- **Manipulação de Processos:** `CreateProcessA`, `OpenProcess`, `TerminateProcess`.
- **Persistência e Registro:** `RegCreateKeyEx`, `RegSetValueEx`.
- **Rede:** `InternetOpen`, `HttpSendRequest`, `Socket`.
- **Evasão/Injeção:** `VirtualAllocEx`, `WriteProcessMemory`, `CreateRemoteThread`.

Guia de Referência: APIs do Windows em Malwares

Ao encontrar estas APIs nas importações de um binário, o analista pode categorizar a ameaça da seguinte forma:

Categoria	APIs Principais	Objetivo do Malware
Spyware / Keylogger	SetWindowsHookEx, GetAsyncKeyState	Monitorar teclado e interceptar senhas.
Downloader / C2	URLDownloadToFile, InternetOpen, HttpSendRequest	Baixar payloads adicionais ou receber comandos remotos.

Exfiltração	FtpPutFile, InternetReadFile, GetClipboardData	Roubar arquivos ou dados da área de transferência.
Dropper / Injeção	VirtualAlloc, WriteProcessMemory, CreateRemoteThread	Alocar memória e injetar código em processos legítimos.
Evasão (Anti-Análise)	IsDebuggerPresent, CheckRemoteDebuggerPresent	Detectar se está sendo analisado por um perito e "autodestruir-se".
Persistência	RegSetValueEx, CreateService	Garantir que o malware inicie junto com o Windows.

Técnica: Process Hollowing (Injeção de Processo)

O objetivo principal desta técnica é a **evasão de defesa**. Como o código malicioso roda dentro de um processo assinado e legítimo, muitas ferramentas de segurança e o Gerenciador de Tarefas do Windows não conseguem distinguir a atividade maliciosa da legítima.

O Fluxo de Execução no Assembly

Ao analisar um malware que utiliza Process Hollowing no Ghidra, você deve procurar por esta sequência específica de chamadas de API:

1. **`CreateProcessA`**: O malware inicia um processo alvo (ex: `calc.exe`) com a flag `CREATE_SUSPENDED` (0x4). O processo nasce "congelado".
2. **`NtUnmapViewOfSection`**: (Opcional, mas comum) Desmapeia a memória original do processo legítimo, deixando-o oco.
3. **`VirtualAllocEx`**: Aloca novo espaço de memória dentro do processo suspenso para receber o código malicioso.
4. **`WriteProcessMemory`**: O malware copia seu próprio código (payload) para o espaço alocado no processo alvo.
5. **`SetThreadContext`**: Altera o registrador `EAX` (ou `RIP` em 64 bits) do processo suspenso para apontar para o novo Ponto de Entrada (Entry Point) do código injetado.
6. **`ResumeThread`**: O processo é "descongelado" e começa a executar o código do malware em vez do código original

1. Qual processo está sendo criado em estado suspenso ao usar a chamada de API CreateProcessA?

Resposta: iexplore.exe

2. Qual é o primeiro endereço virtual onde a função CreateProcessA é chamada?

Imagem de evidência: [Espaço para captura de tela do Ghidra: Endereço do CALL no Listing]

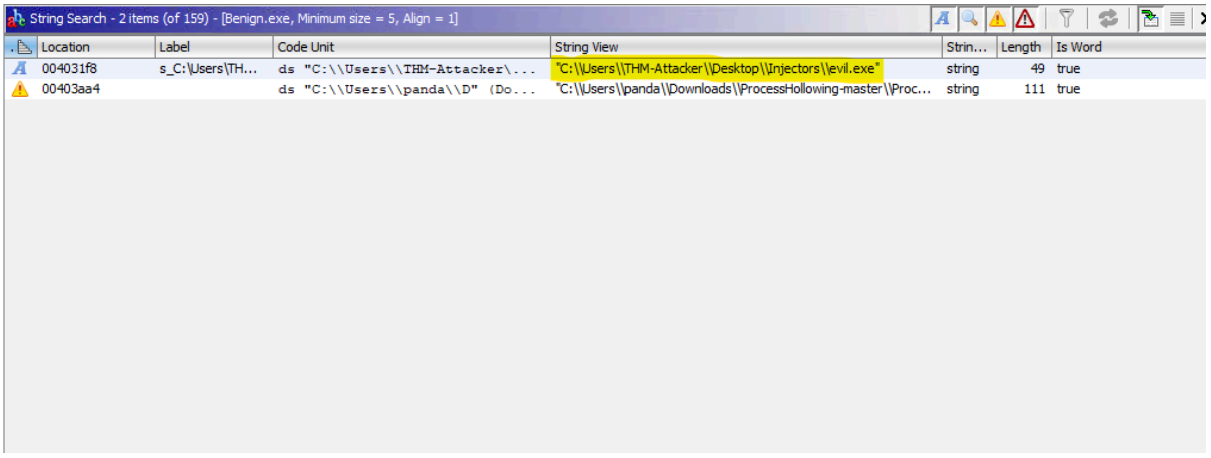
Resposta: 0040108f

```
0040108a 6a 00          PUSH     0x0
0040108c 0f 11 03       MOVUPS   xmmword ptr [EBX],XMM0
0040108f ff 15 34       CALL     dword ptr [->KERNEL32.DLL::Creat
         30 40 00
00401095 85 c0          TEST     EAX,EAX
00401097 75 29          JNZ      LAB_004010c2
00401099 ff 15 18       CALL     dword ptr [->KERNEL32.DLL::GetLa
         30 40 00
0040109f 50             PUSH     EAX
004010a0 68 2c 32       PUSH     s_[-]_Failed_to_create_victim_pr

6)\\Internet Explorer\\iexplore.exe",
RITY_ATTRIBUTES)0x0,0,4,(LPVOID)0x0,
sInformation);

n",(char)DVar3);
DL,in_stack_fffffd14);
```

3. Qual chamada de API é encontrada nas funções de importação usadas para desmapear a memória do processo?

Resposta: ZwUnmapViewOfSection

```
Listing: Benign.exe
00403112 00          ??        00h
00403113 00          ??        00h
                     ****************************************
                     *
                     *  POINTER to EXTERNAL FUNCTION
                     ****************************************
                     undefined NtUnmapViewOfSection()
     undefined          AL:1          <RETURN>
                     660  NtUnmapViewOfSection  <<not bound
                     PTR_NtUnmapViewOfSection_00403114
00403114 88 41 00 00    addr      NTDLL.DLL::NtUnmapViewO
00403118 00          ??        00h
00403119 00          ??        00h
0040311a 00          ??        00h
0040311b 00          ??        00h
```

4. Qual é o caminho completo do processo suspeito mostrado nas strings?

Resposta: C:\\Users\\THM-Attacker\\Desktop\\Injectors\\evil.exe